

LAPORAN PRAKTIKUM
Pemrograman Berorientasi Objek
Relasi Antar Objek (Class Diagram & Asosiasi)
disusun Oleh:

Nama: RIFKI MAULANA

NIM: 2511533007

Kelas: A

DOSEN PENGAMPU : NURFIAH, S.ST, M.Kom



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG, 23 September 2026

A. Langkah Kerja Praktikum

a. Membuat class Diagram Transaksi

1. Buat class Transaksi

```
public class Transaksi {
```

- Class sebagai blueprint dari sebuah objek

2. Buat attribute dan constructor

```
String idTransaksi;  
String jenis;  
double nominal;  
  
// constructor  
public Transaksi(String id, String jenis, double nominal){  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

- Setiap objek memiliki state atau kondisi yang independent. Berfungsi sebagai identitas dan penyimpanan data dari sebuah objek
- Constructor adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya sebagai pemberi nilai awal pada saat sebuah objek pertama kali dibuat. Nanti akan ada objek objek yang dikirim ke nomor, nama, dan saldoAwal yang kemudian dimasukkan ke variable yang telah ditentukan yaitu nomorRekening, namaPemilik, dan saldo.

3. Buat method yang diperlukan

```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | jenis: " + jenis + " | Nominal: Rp" + nominal);  
}
```

- Method ini berfungsi untuk mencetak detail sebuah transaksi dalam satu baris, yaitu menampilkan idTransaksi, jenis, dan nominal.

4. Buka Kembali file Rekening.java dan tambahkan arraylist

```
// Implementasi Asosiasi (1-to-many)
ArrayList<Transaksi> riwayatTransaksi;

public Rekening(String nomor, String nama, double saldoAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    // Wajib menginisialisasi ArrayList di dalam constructor agar tidak NullPointerException
    this.riwayatTransaksi = new ArrayList<>();

    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
}
```

- ArrayList berfungsi untuk menyimpan akun-akun yang dibuat

5. Catat Riwayat transaksi

```
public void setorTunai(double nominal) {
    if (nominal >= 0) {
        saldo += nominal;
        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp " + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}
```

- currentTimeMillis digunakan untuk mendapatkan waktu saat ini atau saat transaksi dibuat
- method ini menambah saldo rekening sekaligus mencatat transaksi ke riwayatTransaksi

```
- package Pekan2;
-
- public class Transaksi {
-
-     String idTransaksi;
-     String jenis;
-     double nominal;
-
-     // constructor
-     public Transaksi(String id, String jenis, double nominal){
-         this.idTransaksi = id;
-         this.jenis = jenis;
-         this.nominal = nominal;
-     }
-     public void cetakDetail() {
-         System.out.println("ID: " + idTransaksi + " | jenis: " + jenis + " | Nominal: Rp" +
-         nominal);
-     }
- }
- }
```

```
- package Pekan2;
- import java.util.ArrayList;
-
- public class Rekening {
-
-     String nomorRekening;
-     String namaPemilik;
```

```

double saldo;

// Implementasi Asosiasi (1-to-many)
ArrayList<Transaksi> riwayatTransaksi;

public Rekening(String nomor, String nama, double saldoAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    // Wajib menginisialisasi ArrayList di dalam constructor agar tidak
    NullPointerException
    this.riwayatTransaksi = new ArrayList<>();

    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan
    saldo Rp" + saldo);
}

public void setorTunai(double nominal) {
    if (nominal >= 0) {
        saldo += nominal;
        // Merekam riwayat (Pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini:
        Rp " + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : minimal nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: saldo tidak mencukupi. Saldo Anda: Rp"
        + saldo);
    } else {
        saldo = saldo - nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini:
        Rp" + saldo);
    }
}

public void cetakMutasi() {
    System.out.println("--- Mutasi Rekening " + nomorRekening + " ---");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Tidak ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }
    }
}

```

```

-     }
-
- }

```

B. Tugas/Latihan praktikum

1. Integrasi Rekam Jejak Penarikan

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : minimal nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
    } else {
        saldo = saldo - nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    }
}

```

2. Fitur Cetak Mutasi Rekening

a. Buat method cetakMutasi()

```

public void cetakMutasi() {
    System.out.println("--- Mutasi Rekening " + nomorRekening + " ---");
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Tidak ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }
    }
}

```

- Method ini berfungsi untuk mencetak mutasi dari rekening. Pertama tama dicek dulu apakah punya riwayatTransaksi kosong atau tidak, kemudian jika tidak kosong maka untuk setiap transaksi di riwayatTransaksi, lakukan method cetakDetail
- b. Tambahkan menu baru 6. Cetak Mutasi (Riwayat)

```
System.out.println("6. Cetak Mutasi (Riwayat");
```

```
case 6:|
    if (akunAktif == null) {
        System.out.println("Error: Anda belum memiliki nomor rekening");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

- Switch case akan melihat terlebih dahulu apakah akunAktif kemudian jika nilainya tidak null maka barulah memanggil method cetakMutasi()